

Department of Electrical and Electronic Engineering
EEEE1027/EEEE1031 Applied Engineering Construction
Project

Report: Semester 2 Project
Raspberry Pi Robot Car (OpenCV)

Name: Euluna Gotami

Student ID: 20113429

Course: Electrical and Electronic Engineering

Contents

Introduction.....	3
Methodology	3
Week 1	3
Assembly.....	3
Basic Movement	4
Move specified distance.....	4
Week 2	4
Sign Detection.....	4
Week 3	5
Move depending on sign.....	5
Face Detection	5
Shape Detection	5
Measuring distance	5
Result/Data Collection & Discussion/Analysis	6
Week 1	6
Results.....	6
Discussion	6
Week 2	6
Results.....	6
Discussion	6
Week 3	7
Move depending on sign.....	7
Face Detection	7
Shape counting.....	7
Measuring Distance	8
Conclusion	9
Reference	9
Appendix.....	10

Introduction

The main goal of this project week is to create a sign detecting robot car that is able to move depending on the sign that it is shown. A real-world application to this would be a self-driving delivery robot that is able to deliver packages to people without human assistance.

Computer vision is used to help computers to do object recognition. Normally, a CPU is needed as an input or output device to process images. However, since the main goal of this project is to create a robot car, a traditional CPU would be too large. Therefore, in this project, a Raspberry Pi is used since it can function as a computer, is low cost, and can be powered by only a single power bank. The program was coded using OpenCV in python which provides real-time computer vision library.

The project was done in a span of three weeks where certain goals/tasks were targeted each week. Furthermore, videos of completion of the tasks were taken and sent to the lecturer as proof of completion.

Week 1 - The goal in week 1 was to do basic movements such as forward, reverse, right, and left. In addition to this, the car was programmed to travel a certain specified distance (29 cm). The GPIO pins on the raspberry pi is used to control the motors.

Week 2 - The goal in week 2 was to have the car recognize symbols shown and give an appropriate output such as labelling the sign or displaying a bounding box around the sign. This is done by using *k-means* and comparing the colours at different locations of the sign.

Week 3 - The goal in week 3 was to have the car move depending on the sign shown by using template matching. In addition to this, since the car is supposed to deliver to humans, the car is programmed to be able to do face detection by using a pre-trained facial recognition model, *Dlib*. The car was also programmed to detect and count shapes, and measure its distance travelled.

Methodology

This section discusses the methods used in the project. The software used for the program was the OpenCV library on Python 3 which is the core of the computer vision of the car.

Week 1

Assembly

Before beginning to code, the car is first assembled as shown in Figure 1.

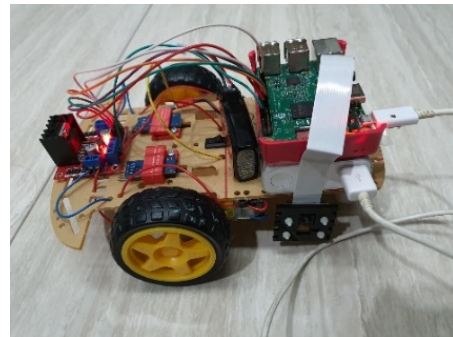


Figure 1. Assembled Raspberry Pi robot car

The hardware used for the car include a Raspberry pi 3B, a Raspberry Pi Camera Rev 1.3, an L298N motor driver, two 5V DC gear motors with wheels, a 900 mAh 7.4V Lipo battery for powering the motors, a 10 000 mAh power bank as its power source, a micro B cable, and some male-female jumper wires to connect the Raspberry pi to the motor driver.

The Raspberry pi 3B comes with an on-board Wireless LAN (WLAN). Since it was not feasible to have a wired connection to a monitor while the car is moving, headless set up was done where the Raspberry Pi is connected wirelessly through Wi-Fi and is able to be accessed and controlled through another computer on the same network.

Basic Movement

To make the car move, a python script is made to control the DC motors. The GPIO library is used. The state of the GPIO output pins on the Raspberry Pi can be changed to high or low to turn the motors on and off. PWM can be changed to adjust the “speed” of the car. In this project, the PWM was set as 40 unless stated otherwise.

Move specified distance

The other task for the week was to make the car move a specified distance. The distance needed to be travelled depended on the last two digits of our student ID. My car needed to travel 29 cm. The time needed for the car to travel that distance was found using trial and error.

Week 2

Sign Detection

For sign detection, five symbols – which were forward, reverse, left, right, and stop – were used.

The code works by first capturing an image from the camera. Then, the image is converted to grayscale since it is necessary for *cv.HoughCircles* – which will look for circles in the image – to work. If a circle is detected, the dominant colour in the circle is checked. The dominant colour of four different zones was then found using *k-means*. *K-means* is a clustering algorithm used in machine learning where

a set of data points are categorised in ‘k’ groups. [1]

If the dominant colour in the circle is red, the sign is a stop sign since it is the only sign that is red. If the dominant colour in the circle is blue, the object is separated into four zones. Below is an example of the code used to assign the zones.

```
zone_0 =
square[square.shape[0]*3//8:square.shape[0] *
5//8, square.shape[1]*1//8:square.shape[1]*3//8]
```

square.shape[0] refers to the vertical axis and *square.shape[1]* refers to the horizontal axis of the box surrounding the circle detected. In the case of the code above, 3/8 to 5/8 of the vertical axis, and 1/8 to 3/8 of the horizontal axis is used for zone 0. Figure 2 shows the placement of zones 0, 1, 2, and 3.

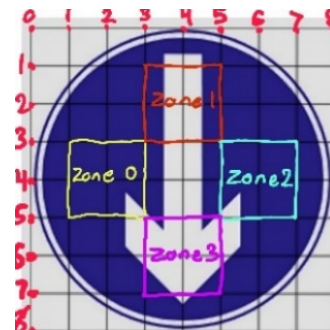


Figure 2. Zones placement

In python, BGR is used instead of RGB. The dominant colours of each zone were then compared. If zone 1 (top) is blue, the sign can only be pointing left or right. The location of the arrowhead is then found by checking if zone 0 (left) or zone 2 (right) is dominant white. If zone 0 (left) is white, arrowhead is pointing left and vice-versa. If zone 1 (top) is not blue, the sign is assumed to be pointing either up or down. The location of the arrowhead is then found by checking if zone 1 (top) or zone 2 (down) is white. If zone 1 (top) is white, arrowhead is pointing top and

vice-versa. Figure 3 shows the flow chart of the code.

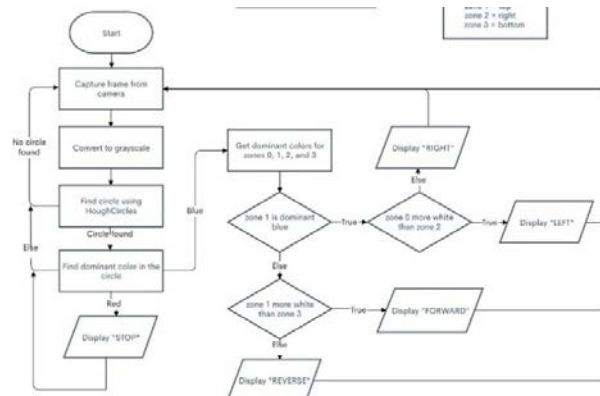


Figure 3: Flowchart of code for symbol detection

Week 3

Move depending on sign

The codes in week 1 (movement) and week 2 (sign recognition) were combined using if statements to make the car move the direction the arrow is pointing at.

Face Detection

OpenCV already contains numerous pre-trained models that can be used by the public. Although it is possible to train an entirely new model, it was not necessary for the task. To train a new face detection model, thousands of positive and negative images are needed. Positive images would be images of faces or images that contain faces. Negative images can be anything else. Collecting the images is time consuming. Thus, a pre-trained model was used.

Shape Detection

The program first captures an image from the camera. The image is then converted to HSV since it is superior to RGB in varying illumination levels. A mask is created from the image captured. The mask is used to only focus on the colour of the shapes. Trackbars are used to adjust the minimum and maximum values

of the range of the Hue, Saturation, and Values until only the target object is visible and the rest black.

The contours are then found by using contour approximation filtering. *arcLength* is used to compute the perimeter of the contour and *approxPolyDP* to obtain the actual contour approximation.

The shapes are identified depending on the number of sides the shape has. Shapes with 3 sides are triangles, 4 sides are squares, and shapes with more than 4 sides are considered circles since the task only required identifying these three shapes.

Measuring distance

For this task, it is necessary to identify the measuring distance symbol. The method used to detect the arrow signs would not work for this symbol since the symbol is not enclosed within a main circle. Thus, template matching was used.

The program first imports the template image (image of symbol) and converts it to grayscale. The program then captures an image from the camera. The captured image is then also converted to grayscale and blurred using *cv2.medianBlur* to help with noise reduction. The template image is then matched to the captured image using *cv2.matchTemplate* which will return a matching value. This matching value – or similarity - can be interpreted as a how similar the most similar object in the captured image is to the template image. If the matching value is above a certain threshold that has been set, the symbol is found in the captured image. Thus, the program will detect the sign. The threshold can be found using trial and error.

Detecting the measuring distance symbol will trigger the car to move forward and start a timer. This timer will stop once the car detects a stop sign. The stop sign is detected using the method in week 2 (dominant colour).

The distance travelled is found using equation 1. The speed can be measured beforehand. However, it is easier to use trial and error.

$$\text{distance} = \text{speed} \times \text{time} \quad (1)$$

PWM of 30 was used in this task.

Result / Data Collection & Discussion / Analysis

This section discusses the results and analysis of the testing done.

Week 1

Results

One of the tasks this week was to make the car do basic movements such as forward, reverse, right, and left. The code works by activating respective GPIO pins for a set number of seconds. The car moved as expected.

The other task was to make the car move 29 cm. Different number of seconds were tested until the car moved exactly 29 cm. The results are shown in Table 1.

Table 1: Test to move car 29 cm

No.	Time (s)	Distance travelled (cm)
1	0.5	13
2	0.6	21
3	0.7	27
4	0.8	29
5	0.9	35

Discussion

The results show that 0.8 seconds was needed to make the car move 29 cm. The

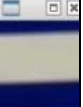
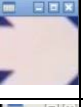
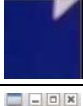
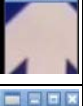
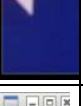
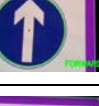
distance travelled by the car increased with the time but varies to some degree (not linear). This is because the motors of the car are easily affected by outside factors such as friction and voltage of the battery – which changes depending on its charge.

Week 2

Results

The task this week was to have the car detect the symbols forward, reverse, left, right, and stop. Results are shown in Table 2.

Table 2: Output of sign detection

	Zone 0	Zone 1	Zone 2	Zone 3	Output
Left					
Right					
Front					
Back					
Stop	X	X	X	X	

Discussion

The results show that the car is able to detect the all the symbols shown to it and label them accordingly. During testing, the label on the symbols sometimes changed to the wrong text. This is because the camera was of low quality and is easily affected by the lighting conditions. The solution was to use a phone to display the signs. The phone acts as its own light source. This led to more consistent readings.

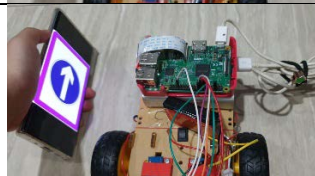
Week 3

Move depending on sign

Results

Different signs were displayed to the car. The outputs of each test were recorded in Table 3.

Table 3: Car movement results

Condition	Direction moved
	Left
	Right
	Forward
	Reverse

Discussion

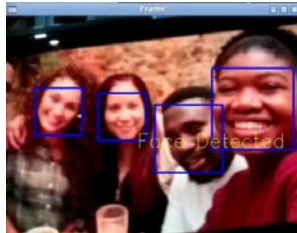
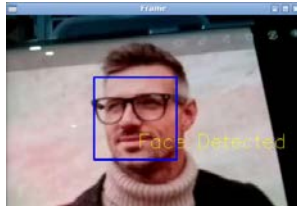
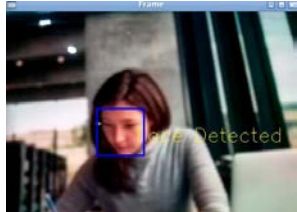
The car was able to move to the direction of the signs. However, much delay (lag) was observed. This is because the method used (*k-means*) requires large computing power. One way to solve this is by using template matching instead. *K-means* method would not be suitable for a moving car since lag of input could cause the car to hit walls.

Face Detection

Results

Various images with faces were shown to the robot. The outputs were compiled in Table 4.

Table 4: Face detection results

Condition	Output
Group setting	
Glasses	
Slightly angled	
Angled	

Discussion

The program was able to detect most of faces shown. The program could not detect the angled face but could still pick up the slightly angled face. This is because the defining features of the face could not be clearly seen. Overall, the program worked quiet well.

Shape counting

Results

The trackbars are used to adjust the lower and upper ranges of hue, saturation, and values. They were adjusted until the mask shows only the shapes we want in white with the rest black.



Figure 4. From left to right, the figure shows the trackbar values, HSV image and mask image.

The values used in the trackbar were noted down and the trackbars were replaced with the values in Figure 4. *arcLength* and *approxPolyDP* were added to the code to obtain the contour approximation. If statements were used to identify the shapes. Figure 5 shows the results.

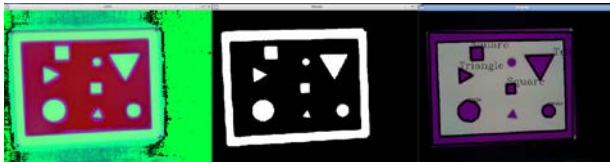


Figure 5. From left to right, the figure shows the HSV image, mask image, and output image.

Discussion

The program had difficulty identifying the smaller shapes. This is because the edges are not as pronounced. This can be improved by using a higher quality camera and increasing the dimensions of the image. However, increasing the image quality would increase the processing power needed to process each image. This would mean that the output would be much more laggy since more time is needed to process each image. This can be solved by using a better CPU compared to the Raspberry pi.

Measuring Distance

Results

The speed of the car needed for calculation was found by using trial and error. The car was put on a straight track

where the 'measuring distance' symbol was put at the 0 cm mark and stop sign at 1.05 m mark (randomly chosen distance). It is known that the car should display 1.05 m. Different values for the *speed* variable were tested until the output showed 1.05 m. Results are shown in Table 5.

Table 5: Testing *speed* variable

Value of <i>speed</i> variable	Output distance (m)
0.1	0.49
0.2	0.95
0.21	1.09
0.22	1.16

0.21 had the nearest value to 1.05 m. Thus, it was used as the *speed* for calculation. The output shown in Figure 6 shows that the distance calculated was 1.10 m. This is close enough to 1.05 m.

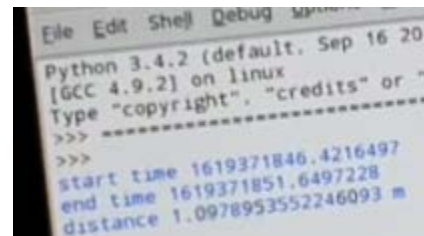


Figure 6. Output for measuring distance

Discussion

Using trial and error, the value of the *speed* was found. The car was then able to calculate the distance using the speed and time using equation 1.

It was observed that the car was able to detect the 'measuring distance' symbol a little faster using the template matching method. This is because template matching uses less computing power.

There was initially difficulty in getting the car to do template matching. It was found that the resolution of the template image had a big impact. Thus, a screenshot of the sign through the raspberry Pi camera

was used instead. This worked much better since it is closer to the resolution of the captured image. The template image used can be seen in Figure 7.

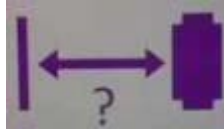


Figure 7. Template image for measuring distance

Conclusion

Sign detection, shape counting, and face detection was done in this project using various OpenCV functions. The project shows that the Raspberry Pi is able to do real-time image processing well even with limited computing power. It was able to do template matching well but struggled with more computing intensive methods such as *k-means*.

Ways to improve the abilities of the car are to use a more powerful computer compared to the Raspberry Pi, or by simply using a better quality camera.

Reference

[1] S. Thakkar, "Dominant colors in an image using k-means clustering", BuzzRobot, 2017. [Online]. Available: <https://buzzrobot.com/dominant-colors-in-an-image-using-k-means-clustering-3c7af4622036>.

Appendix

Week 1 – Move 29 cm

```
import RPi.GPIO as GPIO
import time
```

```
#If IN1=True and IN2=False right motor
move forward, If IN1=False,IN2=True
right motor move backward,in other cases
right motor stop
IN1 = 16 #GPIO23 to IN1 right wheel
direction
IN2 = 18 #GPIO24 to IN2 right wheel
direction
```

```
#If IN3=True and IN4=False left motor
move forward, If IN3=False,IN4=True left
motor move backward,in other cases left
motor stop
IN3 = 13 #GPIO27 to IN3 left wheel
direction
IN4 = 15 #GPIO22 to IN4 left wheel
direction
```

```
#ENA/ENB are PWM(analog) signal pin
which control the speed of right/left motor
through GPIO ChangeDutyCycle(speed)
function
ENA = 12 #GPIO18 to ENA PWM SPEED of
right motor
ENB = 33 #GPIO13 to ENB PWM SPEED of
left motor
```

```
#initialize GPIO pins, tell OS which pins
will be used to control Model-Pi L298N
board
GPIO.setmode(GPIO.BOARD)
GPIO.setup(IN1, GPIO.OUT)
GPIO.setup(IN2, GPIO.OUT)
GPIO.setup(IN3, GPIO.OUT)
GPIO.setup(IN4, GPIO.OUT)
GPIO.setup(ENA, GPIO.OUT)
GPIO.setup(ENB, GPIO.OUT)
```

```
#Initialize ENA and ENB pins, tell OS that
ENA,ENB will output analog PWM signal
with 1000 frequency
rightmotor = GPIO.PWM(ENA,1000)
leftmotor = GPIO.PWM(ENB,1000)
rightmotor.start(0)
leftmotor.start(0)
```

```
#make both motor moving forward
def go_ahead(speed):
    GPIO.output(IN1,True)
    GPIO.output(IN2,False)
    GPIO.output(IN3,True)
    GPIO.output(IN4,False)
```

```
#ChangeDutyCycle(speed) function can
change the motor rotation speed
rightmotor.ChangeDutyCycle(speed)
leftmotor.ChangeDutyCycle(speed)
```

```
#make both motor moving backward
def go_back(speed):
    GPIO.output(IN2,True)
    GPIO.output(IN1,False)
    GPIO.output(IN4,True)
    GPIO.output(IN3,False)
    rightmotor.ChangeDutyCycle(speed)
    leftmotor.ChangeDutyCycle(speed)
```

```
#making left motor moving forward,right
motor no movement
```

```
def turn_left(speed):
    GPIO.output(IN2,False)
    GPIO.output(IN1,False)
    GPIO.output(IN3,True)
    GPIO.output(IN4,False)
    leftmotor.ChangeDutyCycle(speed)
```

```
#make right motor moving forward and
left motor no movement
```

```
def turn_right(speed):
    GPIO.output(IN1,True)
    GPIO.output(IN2,False)
    GPIO.output(IN4,False)
    GPIO.output(IN3,False)
    rightmotor.ChangeDutyCycle(speed)
```

```
#make both motor stop
def stop_car():
    GPIO.output(IN2,False)
    GPIO.output(IN1,False)
    GPIO.output(IN3,False)
    GPIO.output(IN4,False)
```

```
#uncomment to turn right
#turn_right(40)
#time.sleep(.6)
#stop_car()
```

```
#uncomment to turn left
#turn_left(40)
#time.sleep(.6)
#stop_car()
```

```
#uncomment to reverse
#go_back(40)
#time.sleep(.6)
#stop_car()
```

```
go_ahead(40)
time.sleep(.8)
stop_car()
```

```
GPIO.cleanup()
```

Week 2 – Sign detection using k-means

```
import cv2
import numpy as np
from scipy.stats import itemfreq
```

```
def get_dominant_color(image, n_colors):
    pixels = np.float32(image).reshape((-1, 3))
    criteria = (cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER, 200, .1)
    flags =
cv2.KMEANS_RANDOM_CENTERS
```

```
    flags, labels, centroids =
cv2.kmeans(pixels, n_colors, None,
criteria, 10, flags)
    palette = np.uint8(centroids)
    return
palette[np.argmax(itemfreq(labels)[: , -
1])]
```

```
clicked = False
def onMouse(event, x, y, flags, param):
    global clicked
    if event == cv2.EVENT_LBUTTONUP:
        clicked = True
```

```
cameraCapture = cv2.VideoCapture(0)
cv2.namedWindow('camera')
cv2.setMouseCallback('camera', onMouse)
```

```
# Read and process frames in loop
success, frame = cameraCapture.read()
```

```
while success and not clicked:
    cv2.waitKey(1)
    success, frame =
cameraCapture.read()
```

```
    gray = cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY)
    img = cv2.medianBlur(gray, 37)
    circles = cv2.HoughCircles(img,
cv2.HOUGH_GRADIENT,
1, 300, param1=120,
param2=40)
```

```
    if not circles is None:
        circles =
np.uint16(np.around(circles))
        max_r, max_i = 0, 0
        for i in range(len(circles[:, :,
2][0])):
            if circles[:, :, 2][0][i] > 50 and
circles[:, :, 2][0][i] > max_r:
                max_i = i
                max_r = circles[:, :, 2][0][i]
```

```

x, y, r = circles[:, :, :][0][max_i]
if y > r and x > r:
    square = frame[y-r:y+r, x-r:x+r]

    dominant_color =
get_dominant_color(square, 2)
    if dominant_color[2] > 110:
        print("STOP")
        cv2.putText(frame, 'STOP', (x
+ r, y + r),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255,
0), 4)

        elif dominant_color[0] > 70:
            zone_0 =
square[square.shape[0]*3//8:square.shap
e[0]
                * 5//8,
square.shape[1]*1//8:square.shape[1]*3/
/8]
            cv2.imshow('Zone0', zone_0)
            zone_0_color =
get_dominant_color(zone_0, 1)

            zone_1 =
square[square.shape[0]*1//8:square.shap
e[0]
                * 3//8,
square.shape[1]*3//8:square.shape[1]*5/
/8]
            cv2.imshow('Zone1', zone_1)
            zone_1_color =
get_dominant_color(zone_1, 1)

            zone_2 =
square[square.shape[0]*3//8:square.shap
e[0]
                * 5//8,
square.shape[1]*5//8:square.shape[1]*7/
/8]
            cv2.imshow('Zone2', zone_2)
            zone_2_color =
get_dominant_color(zone_2, 1)

```

```

        zone_3 =
square[square.shape[0] * 5 //
8:square.shape[0]
                * 7 // 8,
square.shape[1] * 3 // 8:square.shape[1]
* 5 // 8]
        cv2.imshow('Zone3', zone_3)
        zone_3_color =
get_dominant_color(zone_3, 1)

        if zone_1_color[2] < 50:
            if sum(zone_0_color) >
sum(zone_2_color):
                print("LEFT")

cv2.putText(frame, 'LEFT', (x+r, y+r), cv2.F
ONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 4)
            else:
                print("RIGHT")

cv2.putText(frame, 'RIGHT', (x+r, y+r), cv2.
FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 4)
            else:
                if sum(zone_1_color) >
sum(zone_3_color):
                    print("FORWARD")

cv2.putText(frame, 'FORWARD', (x+r, y+r),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0),
, 4)
            else:
                print("REVERSE")

cv2.putText(frame, 'REVERSE', (x+r, y+r), c
v2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0),
4)
            else:
                print("N/A")

        for i in circles[0, :]:
            cv2.circle(frame, (i[0], i[1]), i[2],
(0, 255, 0), 2)
            cv2.circle(frame, (i[0], i[1]), 2,
(0, 0, 255), 3)

```

```

cv2.imshow('camera', frame)

cv2.destroyAllWindows()
cameraCapture.release()

Week 3 – Move with sign
import cv2
import numpy as np
from scipy.stats import itemfreq

import RPi.GPIO as GPIO
import time

IN1 = 16
IN2 = 18

IN3 = 13
IN4 = 15

ENA = 12
ENB = 33

GPIO.setmode(GPIO.BOARD)
GPIO.setup(IN1, GPIO.OUT)
GPIO.setup(IN2, GPIO.OUT)
GPIO.setup(IN3, GPIO.OUT)
GPIO.setup(IN4, GPIO.OUT)
GPIO.setup(ENA, GPIO.OUT)
GPIO.setup(ENB, GPIO.OUT)

rightmotor = GPIO.PWM(ENA,1000)
leftmotor = GPIO.PWM(ENB,1000)
rightmotor.start(0)
leftmotor.start(0)

def go_ahead(speed):
    GPIO.output(IN1,True)
    GPIO.output(IN2,False)
    GPIO.output(IN3,True)
    GPIO.output(IN4,False)

    rightmotor.ChangeDutyCycle(speed)
    leftmotor.ChangeDutyCycle(speed)

```

```

def go_back(speed):
    GPIO.output(IN2,True)
    GPIO.output(IN1,False)
    GPIO.output(IN4,True)
    GPIO.output(IN3,False)
    rightmotor.ChangeDutyCycle(speed)
    leftmotor.ChangeDutyCycle(speed)

def turn_left(speed):
    GPIO.output(IN2,False)
    GPIO.output(IN1,False)
    GPIO.output(IN3,True)
    GPIO.output(IN4,False)
    leftmotor.ChangeDutyCycle(speed)

def turn_right(speed):
    GPIO.output(IN1,True)
    GPIO.output(IN2,False)
    GPIO.output(IN4,False)
    GPIO.output(IN3,False)
    rightmotor.ChangeDutyCycle(speed)

def stop_car():
    GPIO.output(IN2,False)
    GPIO.output(IN1,False)
    GPIO.output(IN3,False)
    GPIO.output(IN4,False)

def get_dominant_color(image, n_colors):
    pixels = np.float32(image).reshape((-1, 3))
    criteria = (cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER, 200, .1)
    flags =
cv2.KMEANS_RANDOM_CENTERS
    flags, labels, centroids =
cv2.kmeans(pixels, n_colors, None,
criteria, 10, flags)
    palette = np.uint8(centroids)
    return
palette[np.argmax(itemfreq(labels)[:,-1])]

clicked = False

```

```

def onMouse(event, x, y, flags, param):
    global clicked
    if event == cv2.EVENT_LBUTTONDOWN:
        clicked = True

cameraCapture = cv2.VideoCapture(0)
cv2.namedWindow('camera')
cv2.setMouseCallback('camera', onMouse)

# Read and process frames in loop
success, frame = cameraCapture.read()

while success and not clicked:
    cv2.waitKey(1)
    success, frame = cameraCapture.read()

    gray = cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY)
    img = cv2.medianBlur(gray, 37)
    circles = cv2.HoughCircles(img,
cv2.HOUGH_GRADIENT,
                                1, 300, param1=120,
param2=40)

    if not circles is None:
        circles =
np.uint16(np.around(circles))
        max_r, max_i = 0, 0
        for i in range(len(circles[:, :,
2][0])):
            if circles[:, :, 2][0][i] > 50 and
circles[:, :, 2][0][i] > max_r:
                max_i = i
                max_r = circles[:, :, 2][0][i]
            x, y, r = circles[:, :, :][0][max_i]
            if y > r and x > r:
                square = frame[y-r:y+r, x-r:x+r]

                dominant_color =
get_dominant_color(square, 2)
                if dominant_color[2] > 110:
                    print("STOP")
                    cv2.putText(frame, 'STOP', (x
+ r, y + r),

```

```

cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255,
0), 4)

        elif dominant_color[0] > 70:
            zone_0 =
square[square.shape[0]*3//8:square.shap
e[0]
                                * 5//8,
square.shape[1]*1//8:square.shape[1]*3/
/8]
            #cv2.imshow('Zone0', zone_0)
            zone_0_color =
get_dominant_color(zone_0, 1)

            zone_1 =
square[square.shape[0]*1//8:square.shap
e[0]
                                * 3//8,
square.shape[1]*3//8:square.shape[1]*5/
/8]
            #cv2.imshow('Zone1', zone_1)
            zone_1_color =
get_dominant_color(zone_1, 1)

            zone_2 =
square[square.shape[0]*3//8:square.shap
e[0]
                                * 5//8,
square.shape[1]*5//8:square.shape[1]*7/
/8]
            #cv2.imshow('Zone2', zone_2)
            zone_2_color =
get_dominant_color(zone_2, 1)

            zone_3 =
square[square.shape[0] * 5 //
8:square.shape[0]
                                * 7 // 8,
square.shape[1] * 3 // 8:square.shape[1]
* 5 // 8]
            #cv2.imshow('Zone3', zone_3)
            zone_3_color =
get_dominant_color(zone_3, 1)

            if zone_1_color[2] < 50:

```



```

        if sum(zone_0_color) >
sum(zone_2_color):
            print("LEFT")

            turn_left(50)
            time.sleep(.6)
            stop_car()

cv2.putText(frame,'LEFT',(x+r,y+r),cv2.F
ONT_HERSHEY_SIMPLEX,1,(0,255,0),4)
        else:
            print("RIGHT")

            turn_right(50)
            time.sleep(.6)
            stop_car()

cv2.putText(frame,'RIGHT',(x+r,y+r),cv2.
FONT_HERSHEY_SIMPLEX,1,(0,255,0),4)
        else:
            if sum(zone_1_color) >
sum(zone_3_color):
                print("FORWARD")

                go_ahead(40)
                time.sleep(.5)
                stop_car()

cv2.putText(frame,'FORWARD',(x+r,y+r),
cv2.FONT_HERSHEY_SIMPLEX,1,(0,255,0)
,4)
            else:
                print("REVERSE")

                go_back(40)
                time.sleep(.5)
                stop_car()

cv2.putText(frame,'REVERSE',(x+r,y+r),c
v2.FONT_HERSHEY_SIMPLEX,1,(0,255,0),
4)
        else:

```

```

        print("N/A")

        for i in circles[0, :]:
            cv2.circle(frame, (i[0], i[1]), i[2],
(0, 255, 0), 2)
            cv2.circle(frame, (i[0], i[1]), 2,
(0, 0, 255), 3)
            cv2.imshow('camera', frame)

```

```

cv2.destroyAllWindows()
cameraCapture.release()

```

Week 3 – Face detection

```

from imutils import face_utils
import dlib
import cv2
from imutils.video import VideoStream
import imutils
import time

detector =
dlib.get_frontal_face_detector()

print("->Starting Face Detection")
#c = VideoStream(src=0).start()
    #For webcam, comment it if using
Raspberry Pi Camera module
c =
VideoStream(usePiCamera=True).start()
    #For Raspberry Pi Camera module,
comment it if using webcam
time.sleep(2.0)

while True:

    frame = c.read()
    frame = imutils.resize(frame,
width=450)
    gray = cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY)
    rects = detector(gray, 0)

    for rect in rects:
        x1 = rect.left()

```

```

        y1 = rect.top()
        x2 = rect.right()
        y2 = rect.bottom()
        cv2.putText(frame, "Face Detected",
(200, 200),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255,
255))

```

```

        frame = cv2.rectangle(frame,
(x1,y1), (x2,y2), (255, 0, 0), 2)
        cv2.imshow("Frame", frame)

```

```

        key = cv2.waitKey(1) & 0xFF

```

```

        if key == ord("q"):
            break

```

```

cv2.destroyAllWindows()
c.stop()

```

Week 3 – Shape counting

```

import cv2
import numpy as np

```

```

circle = 0
square = 0
triangle = 0
num = 0

```

```

def nothing(x):
    pass

```

```

cap = cv2.VideoCapture(0)

```

```

cv2.namedWindow("Trackbars")
cv2.createTrackbar("L-H", "Trackbars",
46, 180, nothing)
cv2.createTrackbar("L-S", "Trackbars",
90, 255, nothing)
cv2.createTrackbar("L-V", "Trackbars",
39, 255, nothing)
cv2.createTrackbar("U-H", "Trackbars",
178, 180, nothing)
cv2.createTrackbar("U-S", "Trackbars",
255, 255, nothing)

```

```

cv2.createTrackbar("U-V", "Trackbars",
185, 255, nothing)

```

```

font = cv2.FONT_HERSHEY_COMPLEX

```

```

while True:
    _, frame = cap.read()
    hsv = cv2.cvtColor(frame,
cv2.COLOR_BGR2HSV)

```

```

    l_h = cv2.getTrackbarPos("L-H",
"Trackbars")
    l_s = cv2.getTrackbarPos("L-S",
"Trackbars")
    l_v = cv2.getTrackbarPos("L-V",
"Trackbars")
    u_h = cv2.getTrackbarPos("U-H",
"Trackbars")
    u_s = cv2.getTrackbarPos("U-S",
"Trackbars")
    u_v = cv2.getTrackbarPos("U-V",
"Trackbars")

```

```

    lower_red = np.array([l_h, l_s, l_v])
    upper_red = np.array([u_h, u_s, u_v])

```

```

    mask = cv2.inRange(hsv, lower_red,
upper_red)
    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.erode(mask, kernel)

```

```

    _, contours, _ =
cv2.findContours(mask, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)

```

```

    for cnt in contours:
        area = cv2.contourArea(cnt)
        approx = cv2.approxPolyDP(cnt,
0.02*cv2.arcLength(cnt, True), True)
        x = approx.ravel()[0]
        y = approx.ravel()[1]

```

```

        if area > 400:
            cv2.drawContours(frame,
[approx], 0, (0, 0, 0), 5)

```

```

        if len(approx) == 3:
            cv2.putText(frame, "Triangle",
            (x, y), font, 1, (0, 0, 0))
            triangle = triangle + 1
            print("triangle",triangle)

        elif len(approx) == 4:
            cv2.putText(frame, "Square",
            (x, y), font, 1, (0, 0, 0))
            square = square + 1
            print("square",square)
        else:
            cv2.putText(frame, "circle", (x,
            y), cv2.FONT_HERSHEY_COMPLEX, 0.5,
            (0, 0, 0))
            circle = circle + 1
            print("circle",circle)

    cv2.imshow("Frame", frame)
    cv2.imshow("Mask", mask)
    cv2.imshow("HSV", hsv)

    circle = 0
    square = 0
    triangle = 0

    key = cv2.waitKey(1)
    if key == 27:
        break

```

```

cap.release()
cv2.destroyAllWindows()

```

Week 3 – Measure Distance (template used was Figure 7)

```

import cv2
import numpy as np
from scipy.stats import itemfreq
import time

import RPi.GPIO as GPIO
import time

```

```

GPIO.setwarnings(False)
IN1 = 16 #GPIO23 to IN1 right wheel
direction
IN2 = 18 #GPIO24 to IN2 right wheel
direction

IN3 = 13 #GPIO27 to IN3 left wheel
direction
IN4 = 15 #GPIO22 to IN4 left wheel
direction

#ENA/ENB are PWM(analog) signal pin
which control the speed of right/left motor
through GPIO ChangeDutyCycle(speed)
function
ENA = 12 #GPIO18 to ENA PWM SPEED of
right motor
ENB = 33 #GPIO13 to ENB PWM SPEED of
left motor

#initialize GPIO pins, tell OS which pins
will be used to control Model-Pi L298N
board
GPIO.setmode(GPIO.BOARD)
GPIO.setup(IN1, GPIO.OUT)
GPIO.setup(IN2, GPIO.OUT)
GPIO.setup(IN3, GPIO.OUT)
GPIO.setup(IN4, GPIO.OUT)
GPIO.setup(ENA, GPIO.OUT)
GPIO.setup(ENB, GPIO.OUT)

```

```

# Initialize ENA and ENB pins, tell OS that
ENA,ENB will output analog PWM signal
with 1000 frequency
rightmotor = GPIO.PWM(ENA,1000)
leftmotor = GPIO.PWM(ENB,1000)
rightmotor.start(0)
leftmotor.start(0)

```

```

# make both motor moving forward
def go_ahead(speed):
    GPIO.output(IN1,True)
    GPIO.output(IN2,False)
    GPIO.output(IN3,True)
    GPIO.output(IN4,False)

```

```

    #ChangeDutyCycle(speed) function can
    change the motor rotation speed
    rightmotor.ChangeDutyCycle(speed)
    leftmotor.ChangeDutyCycle(speed)

```

```

#make both motor stop

```

```

def stop_car():

```

```

    GPIO.output(IN2,False)

```

```

    GPIO.output(IN1,False)

```

```

    GPIO.output(IN3,False)

```

```

    GPIO.output(IN4,False)

```

```

def get_dominant_color(image, n_colors):

```

```

    pixels = np.float32(image).reshape((-1, 3))

```

```

    criteria = (cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER, 200, .1)

```

```

    flags =

```

```

cv2.KMEANS_RANDOM_CENTERS

```

```

    flags, labels, centroids =

```

```

cv2.kmeans(pixels, n_colors, None,
criteria, 10, flags)

```

```

    palette = np.uint8(centroids)

```

```

    return

```

```

palette[np.argmax(itemfreq(labels)[:,-1])]

```

```

clicked = False

```

```

def onMouse(event, x, y, flags, param):

```

```

    global clicked

```

```

    if event == cv2.EVENT_LBUTTONDOWN:

```

```

        clicked = True

```

```

cameraCapture = cv2.VideoCapture(0)

```

```

cameraCapture.set(3,320)

```

```

cameraCapture.set(4,240)

```

```

cv2.namedWindow('camera')

```

```

cv2.setMouseCallback('camera', onMouse)

```

```

# Read and process frames in loop

```

```

success, frame = cameraCapture.read()

```

```

font = cv2.FONT_HERSHEY_SIMPLEX

```

```

similarity = 0.6

```

```

template_Dis =

```

```

cv2.imread("/home/pi/Desktop/Distance.p
ng", cv2.IMREAD_GRAYSCALE)

```

```

w_Dis, h_Dis = template_Dis.shape[:,-1]

```

```

trigger = 0

```

```

start_trigger = 0

```

```

end_trigger = 0

```

```

while success and not clicked:

```

```

    distance = 0

```

```

    speed = 0.21

```

```

    cv2.waitKey(1)

```

```

    success, frame =

```

```

cameraCapture.read()

```

```

    gray = cv2.cvtColor(frame,

```

```

cv2.COLOR_BGR2GRAY)

```

```

    img = cv2.medianBlur(gray, 37)

```

```

    circles = cv2.HoughCircles(img,

```

```

cv2.HOUGH_GRADIENT,

```

```

                                1, 390, param1=120,

```

```

                                param2=40)

```

```

res_Dis = cv2.matchTemplate(gray,

```

```

template_Dis, cv2.TM_CCOEFF_NORMED)

```

```

loc_Dis = np.where(res_Dis >=
similarity)

```

```

for pt in zip(*loc_Dis[::-1]):

```

```

    cv2.rectangle(frame, pt, (pt[0] +

```

```

w_Dis, pt[1] + h_Dis), (0, 255, 0), 3)

```

```

    cv2.putText(frame,

```

```

'MeasureDistance', (pt[0] + w_Dis, pt[1]
+ h_Dis), font, 1, (0, 255, 0), 1)

```

```

    trigger = 1

```

```

    go_ahead(30)

```

```

if start_trigger == 0:

```

```

    start_trigger = 1

```

```

    start_time = time.time()

```

```

    print("start time", start_time)

```

```

    if not circles is None:
        circles =
np.uint16(np.around(circles))
        max_r, max_i = 0, 0
        for i in range(len(circles[:, :,
2][0])):
            if circles[:, :, 2][0][i] > 50 and
circles[:, :, 2][0][i] > max_r:
                max_i = i
                max_r = circles[:, :, 2][0][i]
            x, y, r = circles[:, :, :][0][max_i]
            if y > r and x > r:
                square = frame[y-r:y+r, x-r:x+r]
                dominant_color =
get_dominant_color(square, 2)

                if dominant_color[2] > 200:
                    stop_car()
                    if trigger == 1:
                        if end_trigger == 0:
                            end_trigger = 1
                            end_time = time.time()
                            distance = speed *
(end_time - start_time)
                            print("end time",
end_time)
                            print("distance", distance,
"m")
                            exit()
                        else:
                            print("STOP")
                            cv2.putText(frame, 'STOP',
(x + r, y + r),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255,
0), 4)

                    for i in circles[0, :]:
                        cv2.circle(frame, (i[0], i[1]), i[2],
(0, 255, 0), 2)
                        cv2.circle(frame, (i[0], i[1]), 2,
(0, 0, 255), 3)

cv2.imshow('camera', frame)

GPIO.cleanup()
GPIO.setwarnings(False)
cv2.destroyAllWindows()
cameraCapture.release()

```